Graduate Team 2: Preventive and Perfective Maintenance in WeeWX

CS 5340 Software Maintenance:
Semester: Spring
Mid-term Presentation

Team members:
Andrew Nguyen
Jerry Delva

Permission is granted for publication on the UCCS website.

Professor: Dr. Armin Moin

GitHub Repository:
https://github.com/anguyen7-99/CS-5340-Group-2.git

Project Overview (Jerry)

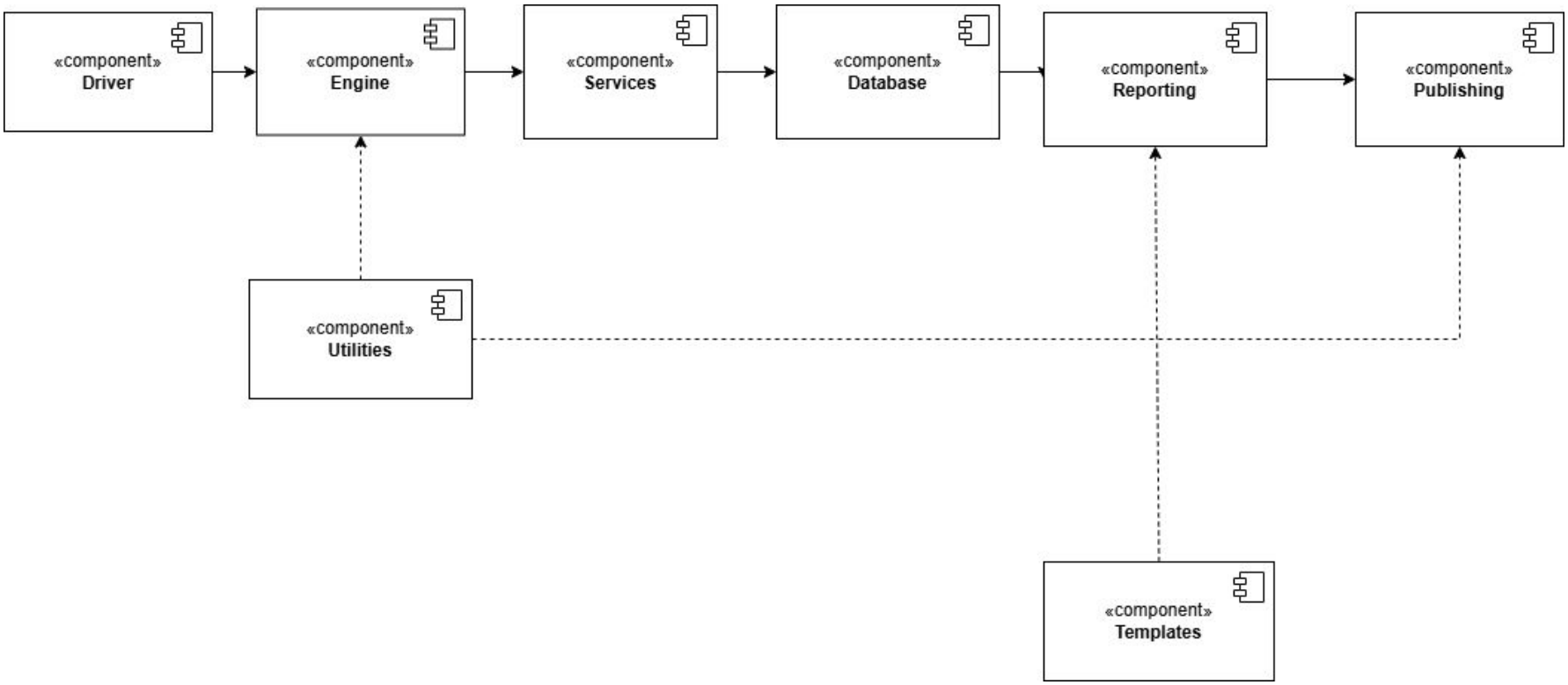
The WeeWX weather system is written in Python and runs on Linux, Mac OS X, Solaris, and BSD.

It collects data from many types of weather stations and sensors, then generates plots, HTML pages, and monthly and yearly summary reports.

It can push plots, pages, and reports to a web server, and data to many online weather services.

- Developed 2008 - 2009
- GitHub Link: https://github.com/weewx/weewx

UML Component Diagram (Andrew)



Maintenance Approach: Preventive and Perfective (Jerry)

- **Preventive:** Multiple collaborators may introduce complexity over time. Using static analysis metrics can help detect complexity early
- **Perfective:** Improve readability and code structure (without changing behavior)
- **Why both?:** Identify risks and keep source code maintainable

End Users & Values (Andrew)

End Users:

- Meteorologists
- Weather hobbyists
- Open-source contributors

Value Proposition:

- Provide quantitative maintainability metrics
- Identify high-risk modules early
- Support preventive and perfective maintenance decisions
- Improve long-term sustainability of the WeeWX codebase

Research Question & Hypothesis (Andrew)

Research Question:

- What measurable maintenance risks exist in WeeWX codebase ?
- Can static code metrics detect maintainability degradation across versions?

Hypothesis:

Applying static code metrics across multiple WeeWX versions will reveal high-risk modules and maintenance trends over time

Related Work (Andrew)

- WeeWx Documentation: WeeWX 5.1
- Cyclomatic Complexity/ Maintainability Index(Andrew):
  - **Coleman, D., Ash, D., Lowther, B., & Oman, P. Using Metrics to Evaluate Software System Maintainability. IEEE Computer , 27(8), 44–49, 1994. DOI: 10.1109/2.303623 .**
  - (Introduces/uses a Maintainability Index–style approach that combines metrics such as cyclomatic complexity to assess and compare maintainability.)
  - **Gill, G. K., & Kemerer, C. F. Cyclomatic Complexity Density and Software Maintenance Productivity. IEEE Transactions on Software Engineering, 17(12), 1991. DOI: 10.1109/32.106988.**
  - (Explains cyclomatic complexity and empirically links it to maintenance productivity—useful justification for why tracking CC matters in software maintenance.)

Related Work (Jerry)

- **Vogelsang, A., Fehnker, A., Huuck, R., & Reif, W.**
  Software Metrics in Static Program Analysis.
  University of Augsburg & NICTA, **2010**.
  (Describes integrating software metric computation directly into static program analysis tools, enabling continuous tracking of maintainability and complexity.)

- **Császár, P. J., & Cserép, M.**
  Assessing Software Metrics and Complexity in Student and Open-Source Projects Using Static Analysis.
  13th International Conference on Applied Informatics, **2026**.
  (Presents empirical results showing how static-analysis-based metrics—CC, Maintainability Index, LCOM—identify problematic modules and code smells in evolving projects.)

- **Karanikiotis, T., Papamichail, M., & Symeonidis, A.**
  Analyzing Static Analysis Metric Trends Toward Early Identification of Non-Maintainable Software Components.
  Sustainability, MDPI, 2021.
  (Shows that tracking metric trends over time—complexity, cohesion, coupling—can identify high-risk, non-maintainable classes early in the software lifecycle.)

Methodology (Andrew)

Static Analysis (Proposed)

Metrics:

- Cyclomatic Complexity: number of possible paths through the code (more branching means higher complexity)
  - Higher complexity is harder to understand, test, and maintain
- Maintainability Index: overall “ease to maintain” score based on code size and complexity
  - Lower score is harder to change safely and is a good refactoring target

Tools:

- Radon: calculates Cyclomatic Complexity and Maintainability Index
- Pylint: reports code quality issues and refactoring opportunities

Method:

- Run the same tools on an older and a current version using consistent settings
- Compare results to find high-risk modules and repeated issues
- Recommend preventive improvements (reduce future risk) and perfective improvements (improve structure and efficiency)

Validation Plan (Jerry)

- Re-run Radon and Pylint using the same scope and settings to confirm results are stable and repeatable.
- Compare an older and current WeeWX release to check that metric trends remain consistent across versions.
- Manually review a small set of low - and high-risk modules to confirm the analysis reflects real maintainability risk.

Relationship to Course Content (Jerry)

- Legacy Systems & Maintenance Challenges
  - Treats WeeWX as an evolving system and highlights how complexity and change over time increase long-term maintenance effort and risk.

- Software Evolution & Reengineering/Refactoring
  - Leverages evolution/reengineering themes to identify maintainability decay and propose refactorings to improve understandability.

- Maintenance Types
  - Applying perfective/preventive maintenance concepts to justify metric driven risk reduction for WeeWX.

Work Plans

Andrew:

- Set up WeeWX locally and select an older and current version to compare (same modules in both)
- Run Radon (Cyclomatic Complexity, Maintainability Index) and Pylint (code smells) and collect outputs
- Summarize and rank maintenance hotspots, then propose preventive (risk reduction) and perfective (structure/readability) improvements

Work Plans

Jerry:

- Document preventive and perfective maintenance rationale for WeeWX
- Review the static analysis outputs (Radon, Pylint) and translate findings into maintainability risks
- Compile related academic research supporting preventive & perfective maintenance
- Create visualizations/tables summarizing complexity trends and code quality issues
- Coordinate final report structure and ensure alignment with course requirements

University of Colorado
Colorado Springs

University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus